

Module 6: Methods

Sarah Foss

Objectives

Able to identify the components of a method, including modifiers, return type, name, and parameters.

Demonstrate how to define and call methods, understanding how parameters and return statements work.

Recognize the concept of method overloading and describe the scope and initialization requirements of local variables within methods.

Utilize methods to encapsulate functionality, promoting clean, organized, and reusable code in their programs.

Methods

A method is a series of program instructions designed to perform a specific task.

Methods allow for reusable code, help organize and simplify the program structure, and make maintaining the code easier.

Defining Methods

Creating a method involves writing the statements and providing a ***method declaration*** with:

- method name (follows the same rules as identifiers)
- list of the inputs (called parameters) and their data types
- the output (return value) type

Syntax:

```
modifier returnValueType methodName(list of parameters) {  
    // Method body;  
    return returnValue;  
}
```

Calling (or invoking) a method involves:

- providing the name of the method
- providing the arguments (inputs) if any
- providing variable to store the output (if any)

Syntax:

```
variable = methodName(list of arguments);
```

Defining a Method

method header



```
public static int min(int x, int y){
```

```
    int result;
```

```
    if(x < y){
```

```
        result = x;
```

```
    }else{
```

```
        result = y;
```

```
    }
```

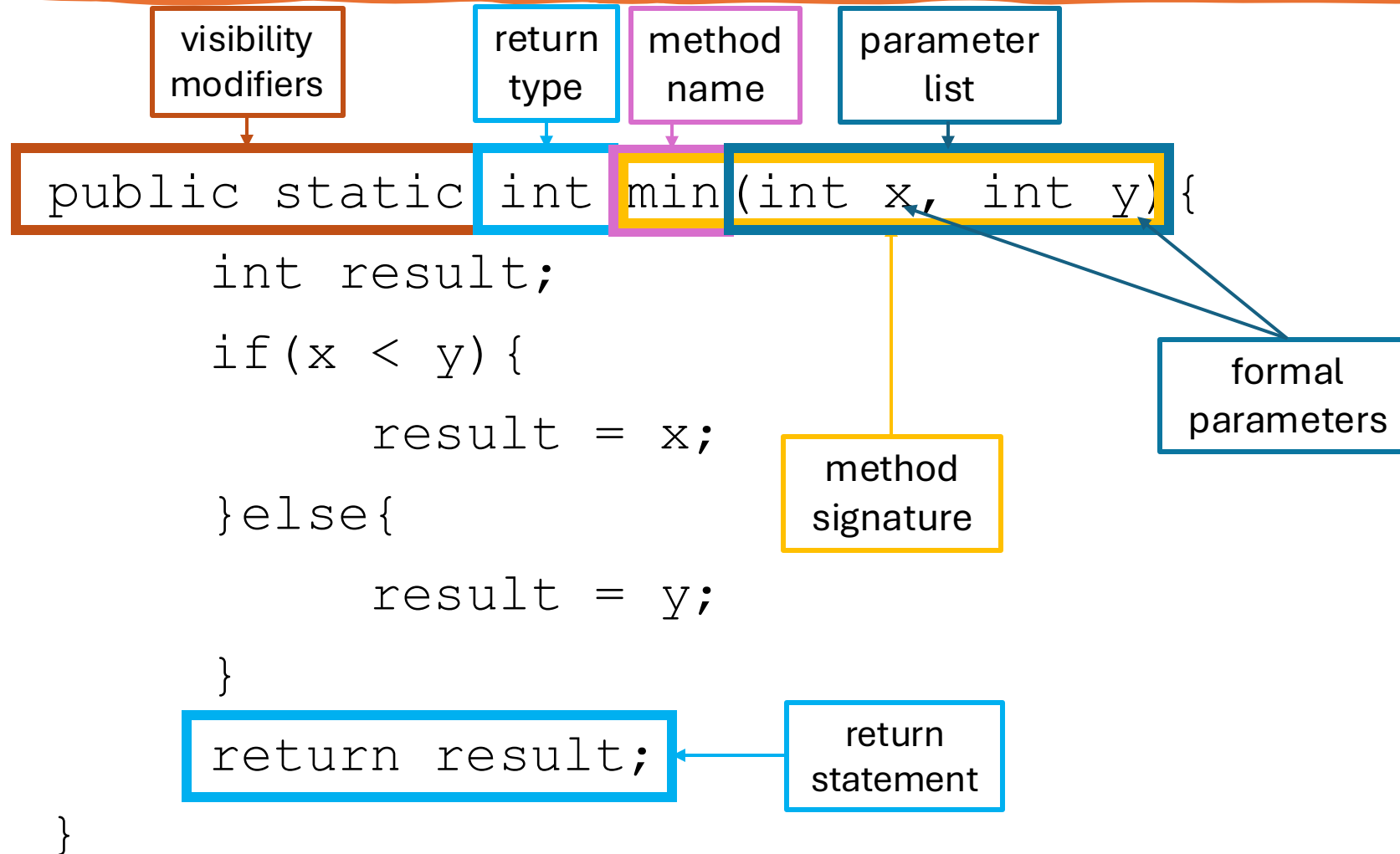
```
    return result;
```

```
}
```

method body

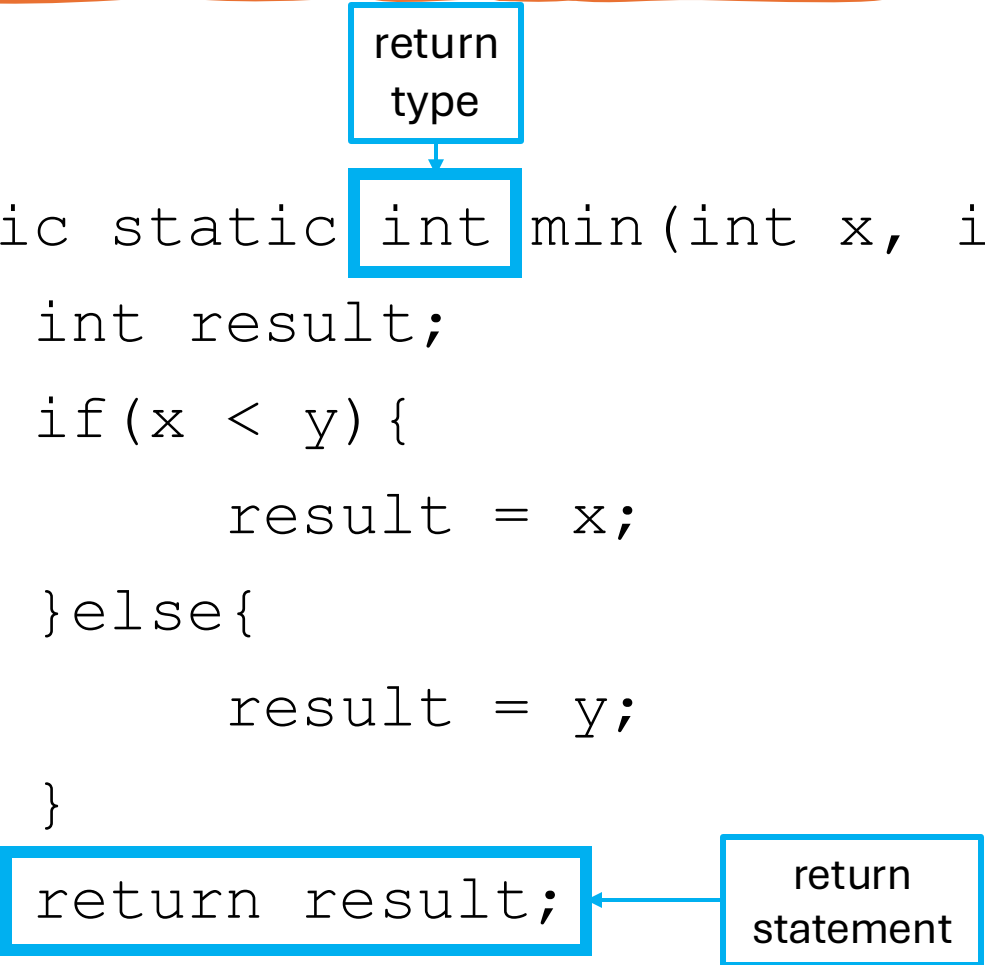


Defining a Method



Returning a Value

```
public static int min(int x, int y) {  
    int result;  
    if(x < y) {  
        result = x;  
    } else {  
        result = y;  
    }  
    return result;  
}
```



If a method is defined with a return type (like `int`, `double`, `String`, etc.), it **must** return a value of that type.

The return statement provides the value back to the method caller.

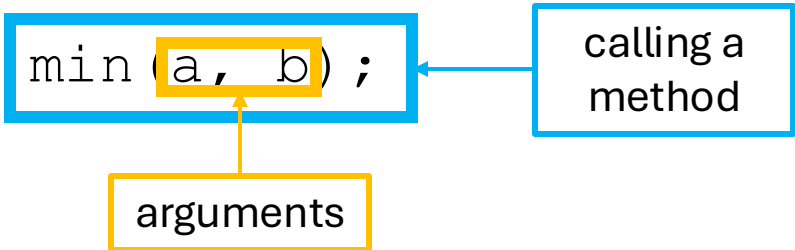
Invoking a Method

```
int a = 20;
```

```
int b = 10;
```

```
int result = min(a, b);
```

calling a
method



arguments

To call (or invoke) a method, you use the method's name followed by parentheses () and pass any required arguments (if needed).

If a method has a return type, you can use the returned value in an expression, assign it to a variable, or output it.

Arguments passed to a method must match the method's parameters in number, type, and order.

Method Notes

When declaring a method, you must put the parenthesis " () " after the name even if the method has no parameters.

Parameter is the term used for input when viewing from inside the method (method's perspective). *Argument* is the term used for input when viewing from outside the method.

Methods are declared only once but can be called as many times as you want.

Execution of the method halts at the return statement and any value in the statement is passed back to the caller.

You may have multiple return statements in a method, but only one will ever be executed for a given execution.

Tracing a Method Call

```
public class TestMin{  
    public static void main(String[] args){  
        int a = 20;  
        int b = 10;  
        int result = min(a, b);  
        System.out.println(result);  
    }  
    public static int min(int x, int y){  
        int result;  
        if(x < y){  
            result = x;  
        }else{  
            result = y;  
        }  
        return result;  
    }  
}
```

If Statement With a Return

Using a return statement in an if-else block lets you return different values based on conditions. Once the return statement executes, the method exits immediately, and no further code runs.

For methods with a return type, a return statement is required. In the method shown in (a), a compilation error occurs because the compiler detects a possible path with no return value.

(a)

```
public String checkNumber(int num) {  
    if (num > 0) {  
        return "Positive";  
    } else if (num < 0) {  
        return "Negative";  
    } else if (num == 0) {  
        return "Zero";  
    }  
}
```

Should
be

(b)

```
public String checkNumber(int num) {  
    if (num > 0) {  
        return "Positive";  
    } else if (num < 0) {  
        return "Negative";  
    } else {  
        return "Zero";  
    }  
}
```

Call Stack

The **call stack** (aka: execution stack, runtime stack, machine stack, or just “the stack.”) is an area of memory that keeps track of the sequence of method calls in a program.

It helps manage the flow of control by storing information about active methods (those that have been called but haven’t finished executing yet).

Each time a method is invoked, a new entry that stores parameters and variables for the method, called a **stack frame** (or **activation record**), is added to the call stack.

When the method completes, the corresponding stack frame is removed.

The call stack operates on a last-in, first-out (LIFO) principle, meaning the most recently added stack frame is the first to be removed when its method finishes.

void Methods

A `void` method is a method that does not return any value.

It performs an action, such as printing text, modifying data, or interacting with other objects, but unlike methods with return types, it doesn't send any value back to the caller when it completes.

```
public static void printGrade(double score) {  
    if (score >= 90.0) {  
        System.out.println('A');  
    } else if (score >= 80.0) {  
        System.out.println('B');  
    } else if (score >= 70.0) {  
        System.out.println('C');  
    } else if (score >= 60.0) {  
        System.out.println('D');  
    } else {  
        System.out.println('F');  
    }  
}
```

void Methods

Although `void` methods do not return a value, they can still use a `return` statement to exit early if necessary.

```
public static void printGrade(double score) {  
    if (score < 0 || score > 100) {  
        System.out.println("Invalid score");  
        return;  
    }  
    if (score >= 90.0) {  
        System.out.println('A');  
    } else if (score >= 80.0) {  
        System.out.println('B');  
    } else if (score >= 70.0) {  
        System.out.println('C');  
    } else if (score >= 60.0) {  
        System.out.println('D');  
    } else {  
        System.out.println('F');  
    }  
}
```

Pass-By-Value

In Java, when a value is passed to a method, it is always **passed-by-value**.

This means the method receives a copy of the value, not the original variable.

As a result, any changes made to the copied value inside the method do not impact the original variable in the calling code.

Pass-By-Value Example

```
public class PassByValueExample {  
    public static void modifyValue(int num) {  
        num = 10;  
        System.out.println(num); //output: 10  
    }  
    public static void main(String[] args) {  
        int original = 5;  
        modifyValue(original);  
        System.out.println(original); //output: 5  
    }  
}
```


Overloading Methods

Method overloading allows a class to have more than one method with the same name, as long as their parameter lists (number, types, or order of parameters) are different.

The Java compiler determines which method to use based on the method signature.

Overloaded methods can have different return types, but it is not required.

Overloading Methods Example

```
public static int min(int x, int y) {  
    if (x < y) {  
        return x;  
    } else {  
        return y;  
    }  
}  
  
public static double min(double x, double y) {  
    if (x < y) {  
        return x;  
    } else {  
        return y;  
    }  
}
```

Local Variable Scope

A **local variable** is a variable that is declared within a method, constructor, or block (such as within a loop or an if statement).

These variables are only visible and usable within the block in which they are defined.

The scope of a local variable begins at the point where it is declared and ends when the enclosing method, constructor, or block is completed.

Once the execution moves outside of the block, the variable is no longer accessible, and its memory is released.

```
public class ScopeExample {  
    public static void main(String[] args) {  
        String mainMsg = "Only accessible in the main method";  
        System.out.println(mainMsg);  
        showMsg1();  
        showMsg2();  
    }  
    public static void showMsg1() {  
        //System.out.println(mainMsg); // Compile error  
        System.out.println("Cannot access variables from main.");  
    }  
    public static void showMsg2() {  
        String msg2 = "Only accessible in method2";  
        System.out.println(msg2);  
    }  
}
```

Method Abstraction

Method abstraction is a concept that involves separating the **implementation** of a method from its **use**.

It means that the details of how a method performs its task are encapsulated in the method and are hidden from the user, who only needs to know what the method does, not how it does it. This is also known as **information hiding** or **encapsulation**.

If you decide to change the implementation, the client program will not be affected, provided you do not change the method signature.

Benefits of Methods

- **Reusability:** Avoid repeating code, improving efficiency.
- **Modularity:** Break complex tasks into manageable pieces and separates concerns.
- **Maintainability:** Simplify updates, debugging, and error handling.
- **Readability:** Make code easier to understand and follow.
- **Abstraction:** Hide implementation details from the client program.

Summary

- Methods help create reusable and organized code.
- A method's header includes modifiers, return type, name, and parameters.
- Methods can return a value, with the return type specifying the data type. If no value is returned, the method uses the void keyword.
- The method signature consists of the method name and parameters. Methods may or may not have parameters.
- Arguments passed to a method must match the method's parameters in number, type, and order.

Summary

- When a method is called, control passes to it and returns to the caller when it finishes or encounters a return statement.
- Value-returning methods can be invoked without using the return value, and methods can be overloaded if they have different parameter lists.
- Local variables are declared within methods and must be initialized before use. Their scope is limited to their block.
- Method abstraction hides the implementation from the client, promoting encapsulation.
- Modular programs are easier to write, debug, and maintain, enhancing reusability.